

## Task - 2

### → The Four "Danger Joins" (Duplication & Temporal Leakage)

The prompt requires explicitly identifying joins that will ruin the metrics if handled improperly. Here is how you document them:

#### 1. Transaction ↔ Fraud Decision (Row Explosion):

- Risk: A single transaction triggers multiple fraud rules (e.g., VELOCITY\_CHECK and DEVICE\_IP\_RISK). Joining directly on transaction\_id will duplicate the transaction row for every rule fired, inflating attempted volume and gross payment value.
- Fix: Pre-aggregate fraud\_decision to the transaction\_id grain (e.g., extracting the MAX(risk\_score) or counting the number of decline decisions) before joining.

#### 2. Transaction ↔ Payment Event (Row Explosion & Temporal Leakage):

- Risk: Every transaction has multiple lifecycle stages (INITIATED, ROUTED, COMPLETED). A direct join duplicates volume. Furthermore, using ingestion\_time instead of event\_time introduces temporal leakage, as events arriving days late will shift transactions into the wrong reporting period.
- Fix: Pre-aggregate to find the MAX(event\_time) and SUM(processing\_ms) per transaction\_id.

#### 3. Transaction ↔ Settlement (Row Explosion):

- Risk: Some transactions are settled in multiple parts (e.g., PARTIAL and FINAL statuses). Joining directly will double-count the original transaction amount.
- Fix: Pre-aggregate settlement\_amount and fee\_amount by transaction\_id before joining to the core transaction table.

#### 4. Transaction ↔ Merchant Risk Snapshot (Fan-Out & Temporal Leakage):

- Risk: The snapshot table contains a risk score for every month. If you join only on merchant\_id, a single January transaction will replicate for January, February, and March snapshots.
- Fix: You must join on merchant\_id AND ensure the transaction's created\_at month perfectly matches the snapshot\_month.

### 3. KPI Date Logic

The prompt requires stating which date should drive each major KPI and why.

Driving Date: created\_at (from transaction.csv)

- Why? Executive KPIs and contribution profitability must be evaluated based on when the actual business activity (the payment attempt) occurred.
- If we use settlement\_date, we artificially deflate profitability in the current period due to standard processing lags (T+2/T+3 settlements).
- If we use ingestion\_time from the event table, we fall victim to the data engineering delays, masking true operational performance. created\_at represents the true moment of commercial intent and operational load.